FRAMEWORK_SELECTION

FRAMEWORK SELECTION — EP-AGENT-001

Version: v0.2-DRAFT (2026-10-15). Versions, release dates, and licenses were verified against authoritative sources (PyPI JSON API, GitHub releases/repository metadata, official documentation) on 2026-10-15 — see §2a. All sources agree on every selected version. Versions must still be pinned in `requirements.lock` at freeze.

---

1. Selection criteria

#	Criterion	Meaning
C1	Active maintenance	Current release within ~90 days; not in maintenance mode
C2	Enterprise relevance	Material enterprise adoption or official vendor backing
C3	Open license / free to install	OSI license; no paid subscription needed for the study
C4	Distinct tool-invocation model	Adds a meaningfully different integration point vs. the other selections
C5	Clean single-agent isolation	Multi-agent / delegation features can be left unused or disabled without distorting the framework, so the study stays out of the UW P2 lane

2. Selection matrix

Field	OpenAI Agents SDK	LangGraph	CrewAI	MAF
Package	`openai-agents`	`langgraph`	`crewai`	`agent-framework`
Version	0.24.0	1.2.15	1.15.28	1.22.0
Release date (PyPI upload, UTC)	2026-10-15	2026-10-12	2026-10-14	2026-10-15
License	MIT	MIT	MIT	MIT
Language/runtime	Python ≥3.10	Python ≥3.10	Python ≥3.10,<3.14	Python ≥3.10 (.NET also available; Python used)
Install	`pip install openai-agents`	`pip install langgraph`	`pip install crewai`	`pip install agent-framework`
Tool invocation model	`@tool`-decorated functions; model-driven reasoning loop	`ToolNode` graph nodes; conditional-edge routing; state machine	Role-based agent tools (`BaseTool`); JSON-first task/crew config	`FunctionMiddleware` wraps every tool call; `call_next` delegate
External API integration	Via tool functions	Via tool nodes	Via agent tools	Via function middleware
Enterprise relevance	HIGH	HIGH	MODERATE-HIGH	HIGH
Maintenance	Active	Active	Active	Active
Delegation features in scope?	NO — excluded (handoffs)	NO — excluded (multi-agent graphs)	NO — excluded (crew delegation)	NO — excluded (multi-agent orchestration)

Field	OpenAI Agents SDK	LangGraph	CrewAI	MAF
Rationale	Widest enterprise adoption; baseline	Industry standard for stateful regulated workflows; different graph-based integration model	Role-based paradigm; meaningfully different config approach	Governance-first; FunctionMiddleware maps cleanly to ExecutionProof call point

2a. Authoritative-source verification (verified 2026-10-15)

Field	OpenAI Agents SDK	LangGraph	CrewAI	Microsoft Agent Framework	Google ADK (future extension — NOT in v1)
Official name	OpenAI Agents SDK	LangGraph	CrewAI	Microsoft Agent Framework	Agent Development Kit (ADK)
Docs URL	https://openai.github.io/openai-agents-python/	https://docs.langchain.com/oss/python/langgraph/overview	https://docs.crewai.com	https://aka.ms/agent-framework	https://google.github.io/adk-docs/
Repo URL	https://github.com/openai/openai-agents-python	https://github.com/langchain-ai/langgraph	https://github.com/crewAIInc/crewAI	https://github.com/microsoft/agent-framework (`python/`)	https://github.com/google/adk-python
Registry URL	https://pypi.org/project/openai-agents/	https://pypi.org/project/langgraph/	https://pypi.org/project/crewai/	https://pypi.org/project/agent-framework/	https://pypi.org/project/google-adk/
Version (PyPI latest)	0.24.0	1.2.15	1.15.28	1.22.0	2.12.0
PyPI upload (UTC)	2026-10-15T02:38:18Z	2026-10-12T22:38:11Z	2026-10-14T04:47:37Z	2026-10-15T09:54:59Z	2026-10-13T16:42:55Z
GitHub release tag / published (UTC)	v0.24.0 / 2026-10-15T02:20:12Z	1.2.15 / 2026-10-12T22:38:26Z	1.15.28 / 2026-10-14T04:47:00Z	python-1.22.0 / 2026-10-15T09:53:40Z	v2.12.0 / 2026-10-13T16:35:19Z
License	MIT (PyPI classifier + GitHub)	MIT (GitHub; PyPI license field empty)	MIT (GitHub; PyPI license field empty)	MIT	Apache-2.0
Language/ runtime	Python ≥3.10	Python ≥3.10	Python ≥3.10,<3.14	Python ≥3.10	Python ≥3.11
Repo archived?	No	No	No	No	No
Reason selected	Baseline in-loop function tool; widest adoption	Graph/state-machine node idiom	Role/task/crew object model	Middleware interceptor idiom	ELIGIBLE FUTURE EXTENSION / NOT INCLUDED IN v1 / NO CHANGE TO 20-CELL MATRIX (see §5)
Date verified	2026-10-15	2026-10-15	2026-10-15	2026-10-15	2026-10-15

Discrepancy check: no source disagreement on any version. Two **release-date metadata corrections** versus v0.1: MAF Python 1.22.0 was released 2026-10-15 (v0.1 listed 2026-10-08; the .NET `dotnet-1.22.0` tag dates from 2026-09-18 and is not the Python package), and CrewAI 1.15.28 was released 2026-10-14 (v0.1 listed “(current)”). Selected versions are unchanged.

3. Criteria evaluation

Criterion	OpenAI Agents SDK	LangGraph	CrewAI	MAF
C1 Active maintenance	✔ release 2026-10-15	✔ release 2026-10-12	✔ release 2026-10-14	✔ release 2026-10-15
C2 Enterprise relevance	✔ HIGH — official OpenAI SDK	✔ HIGH — LangChain ecosystem, regulated workflows	✔ MODERATE-HIGH — popular for rapid prototyping	✔ HIGH — official Microsoft successor to AutoGen; Azure
C3 Open license	✔ MIT	✔ MIT	✔ MIT	✔ MIT
C4 Distinct model	✔ Model-loop function tools (baseline)	✔ Explicit graph/state-machine node	✔ Role/task/crew configuration object model	✔ Middleware interception pattern
C5 Single-agent isolation	✔ Handoffs are opt-in; not configured	✔ Single-node graph; no sub-agents	✔ One agent, one task, `allow_delegation=False`	✔ One agent; middleware on one function tool

All four satisfy C1–C5. Together they cover four distinct integration idioms: in-loop tool, graph node, role-bound tool, and middleware interceptor — which is what makes a portability result informative.

4. Per-framework integration point (single-agent only)

Framework	Where the adapter sits	ExecutionProof call point
OpenAI Agents SDK	`EPVerificationTool` wraps the target action as a function tool	Tool body: normalize → `/verify` → only on ALLOW invoke mock action; return verdict + ProofRecord ref
LangGraph	`EPVerifiedToolNode` provides the tool executed by `ToolNode`	Tool execution inside the node; verdict + ref written into graph state / ToolMessage
CrewAI	`EPVerifiedCrewTool` (`BaseTool` subclass) assigned to one agent	`_run`: normalize → `/verify` → gated mock action; result string carries verdict + ref
MAF	`EPVerificationMiddleware` (`FunctionMiddleware`)	Before `call_next`: normalize → `/verify`; `call_next` only on ALLOW; verdict + ref attached to function result

Exact framework class/method signatures are validated against the pinned versions at freeze; the stubs in `adapters/` do not import framework packages.

All four selected frameworks run v1 with exactly one agent and exactly one tool call per run; single-agent parallel tool calls are disabled or unused (e.g. `parallel_tool_calls=False` in OpenAI Agents SDK model settings; one tool call per turn in LangGraph, CrewAI, and MAF).

5. Other candidates

Candidate	Status	Determination
Google ADK (Python 2.12.0)	Active; Apache-2.0; satisfies C1–C4	**NOT EXCLUDED — ELIGIBLE CANDIDATE (C5 satisfied by configuration).** v0.1 deferred ADK because it supports A2A. Supporting A2A is not by itself a reason to exclude a framework. Evidence from the 2.12.0 wheel and the v2.12.0 `pyproject.toml`: (1) A2A is an optional extra (`google-adk[a2a]` → `a2a-sdk`); the base package does not install it, and no core module

Candidate	Status	Determination
		imports a2a outside `google/adk/a2a/`; (2) agent transfer targets are built only from `sub_agents`, a parent agent, and peers (`flows/llm_flows/extensions/_agent_transfer.py`), so a lone `LlmAgent` with no sub-agents and no parent gets no transfer tool; `disallow_transfer_to_parent` / `disallow_transfer_to_peers` can also be set True; (3) `before_tool_callback` can return a dict that replaces tool execution — a clean ExecutionProof interception point. Strict single-agent, single-tool configuration: one `LlmAgent`, one `FunctionTool`, no `sub_agents`, both disallow-transfer flags True, base package without `[a2a]`, no `RemoteA2aAgent` / `to_a2a`. **Decision recorded (Derek Hone, 2026-10-15): KEEP v1 AT FOUR FRAMEWORKS; Google ADK is NOT added to v1.** ADK status: **ELIGIBLE FUTURE EXTENSION / NOT INCLUDED IN v1 / NO CHANGE TO 20-CELL MATRIX** (Derek Hone decision, 2026-10-15: keep v1 at four frameworks). All P1–P5 denominators stay at 20.
AutoGen	Legacy; maintenance mode as of Oct 2025	Fails C1. Superseded by Microsoft Agent Framework.
AG2 (community AutoGen fork)	Considered	MAF preferred: official vendor backing and higher enterprise relevance (C2); its multi-agent conversation core also weakens C5.

6. Replacement rule

If a selected framework becomes unusable **before** execution (yanked release, critical install defect, license change), it may be replaced only via a written amendment, CHANGELOG entry, regenerated manifest, new preregistration SHA, and a re-run of the 10-question gate. Any replacement must satisfy C1–C5 under the strict single-agent, single-tool configuration; Google ADK is an eligible future extension (not in v1) under that configuration. Failure **during** execution is preserved per `PASS_FAIL_RULES.md`.